UNIVERSIDADE DO ESTADO DE SANTA CATARINA – UDESC

CENTRO DE CIÊNCIAS TECNOLÓGICAS

TRABALHO FINAL – LINGUAGENS FORMAIS A AUTÔMATOS

Alunos: Guilherme Augusto Santos, Arthur Bertoli Silva

Introdução

O objetivo deste trabalho é implementar um Autômato com Pilha (PDA) utilizando a linguagem de programação C. Autômatos com Pilha são modelos computacionais que utilizam uma pilha para gerenciar operações, tornando-os capazes de reconhecer uma classe de linguagens maior do que os autômatos finitos, especificamente as linguagens livres de contexto. O código desenvolvido permite a criação de estados, transições e verificação de palavras com base na definição do autômato, além de gerar um gráfico representativo das transições configuradas. O programa é interativo, proporcionando uma interface de menu que facilita a interação com as funcionalidades do autômato.

Desenvolvimento

Para desenvolver o trabalho, que foi feito na linguagem de programação C, foram usadas diversas funções, para dentro da main chamar e montar as palavras, além de apresentar grafos dos autômatos.

Primeiramente, incluimos as bibliotecas necessárias e definimos o número máximo de estados, transições, pilhas e fitas para cem:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_ESTADOS 100
#define MAX_TRANSICOES 100
#define MAX_PILHA 100
#define MAX_FITA 100
```

Em seguida, criamos as structs de estado, transição, autômato e pilha, respectivamente. Inicializamos as structs automato e pilha:

```c
typedef struct {
    char nome[10];
    int inicial;
    int final;
} Estado;

typedef struct {
    Estado *origem;
    Estado *destino;
    char simbolo;
    char desempilha;
    char empilha;
} Transicao;

typedef struct {
    Estado estados[MAX_ESTADOS];
    int numEstados;
    Estado *estadoInicial;
    Estado *estadosFinais[MAX_ESTADOS];
    int numFinais;
    Transicao transicoes[MAX_TRANSICOES];
    int numTransicoes;
    char alfabetoEntrada[MAX_ESTADOS];
    char alfabetoPilha[MAX_ESTADOS];
} Automato;

typedef struct {
    char pilha[MAX_PILHA];
    int topo;
} Pilha;

Automato automato;
Pilha pilha;
```

void inicializaPilha():

Essa função tem inicializa a pilha. Define o valor inicial do topo como -1, indicando que a pilha está vazia.

```c
void inicializaPilha() {
    pilha.topo = -1;
}
```

void empilhar():

Essa função adiciona um simbolo à pilha. Antes de adicionar , verifica se a pilha está cheia ou não, se houver espaço, ela incrementa o valor do topo e armazena o novo símbolo na posição correspondente.

```
void empilhar(char c) {
    if (pilha.topo < MAX_PILHA - 1) {
        pilha.pilha[++pilha.topo] = c;
    }
}
```

char desempilhar():

Essa função remove e retorna o símbolo para o topo da pilha. Antes de remover, verifica se a pilha está não está vazia, se não estiver, ela decrementa o valor do topo e retorna o símbolo removido. Se a pilha estiver vazia, ela retorna o caractere nulo.

```
char desempilhar() {
    if (pilha.topo >= 0) {
        return pilha.pilha[pilha.topo--];
    }
    return '\0';
}
```

void CriaEstado(char *nome, int inicial, int final):

Essa função cria um novo estado no autômato e o adiciona na lista de estados. Define se é estado inicial e/ou final.

```
void CriaEstado(char *nome, int inicial, int final) {
    Estado e;
    strcpy(e.nome, nome);
    e.inicial = inicial;
    e.final = final;
    automato.estados[automato.numEstados++] = e;
    if (inicial) {
        automato.estadoInicial = &automato.estados[automato.numEstados - 1];
    }
    if (final) {
        automato.estadosFinais[automato.numFinais++] = &automato.estados[automato.numEstados - 1];
    }
}
```

void CriaTransicao(char *origem, char *destino, char simbolo, char desempilha, char empilha):

Essa função cira uma nova transição entre dois estados no autômato. Localiza os estados de origem e destino pelo nome, define os símbolos de transição, desempilhamento e empilhamento, e adiciona a nova transição à lista de transições do autômato.

```
void CriaTransicao(char *origem, char *destino, char simbolo, char desempilha, char empilha) {
    Transicao t;
    for (int i = 0; i < automato.numEstados; i++) {
        if (strcmp(automato.estados[i].nome, origem) == 0) {
            t.origem = &automato.estados[i];
        }
        if (strcmp(automato.estados[i].nome, destino) == 0) {
            t.destino = &automato.estados[i];
        }
    }
    t.simbolo = (simbolo == '-') ? '\0' : simbolo;
    t.desempilha = (desempilha == '-') ? '\0' : desempilha;
    t.empilha = (empilha == '-') ? '\0' : empilha;
    automato.transicoes[automato.numTransicoes++] = t;
}
```

void mostraFuncPrograma():

Essa função exibe todas as transições do autômato. Ela percorre a lista de transições e imprime a função programa.

```
void mostraFuncPrograma() {
    printf("Função Programa (Transições):\n");
    for (int i = 0; i < automato.numTransicoes; i++) {
        printf("δ(%s, %c, %c) = (%s, %c)\n", automato.transicoes[i].origem->nome,
               (automato.transicoes[i].simbolo == '\0') ? 'ε' : automato.transicoes[i].simbolo,
               (automato.transicoes[i].desempilha == '\0') ? '-' : automato.transicoes[i].desempilha,
               automato.transicoes[i].destino->nome,
               (automato.transicoes[i].empilha == '\0') ? '-' : automato.transicoes[i].empilha);
    }
}
```

int leFitaRec(estado *estadoAtual, char*fita, int pos):

Essa função percorre recursivamente as transições do autômato, vendo se a fita é aceita, utiliza as operações de desempilhamento e empilhamento para manipular a pilha e verifica a aceitação com base em alcançar um estado final e ter uma pilha vazia ao fim da leitura da fita.

```
int leFitaRec(Estado *estadoAtual, char *fita, int pos) {
    if (pos == strlen(fita)) {
        for (int i = 0; i < automato.numFinais; i++) {
            if (automato.estadosFinais[i] == estadoAtual && pilha.topo == -1) {
                return 1; // Aceita
            }
        }
    }

    char simbolo = fita[pos];
    for (int i = 0; i < automato.numTransicoes; i++) {
        Transicao t = automato.transicoes[i];
        if (t.origem == estadoAtual && (t.simbolo == simbolo || t.simbolo == '\0')) {
            if (t.desempilha == '\0' || desempilhar() == t.desempilha) {
                if (t.empilha != '\0') {
                    empilhar(t.empilha);
                }
                int novoPos = t.simbolo == '\0' ? pos : pos + 1;
                if (leFitaRec(t.destino, fita, novoPos)) {
                    return 1;
                }
                if (t.empilha != '\0') {
                    desempilhar();
                }
            }
        }
    }
    return 0;
}
```

void plot():

Essa função gera um arquivo DOT que descre a estrutura de um atômato com pilha. O arquivo inclui nós para os estados, arestas para as transições e rótulos que indicam os símbolos de transição e as operações da pilha. Este arquivo pode ser utilizado para visualizar o autômato utilizando ferramentas de visualização de grafos como o Graphviz.

```c
void plot() {
    FILE *file = fopen("pda_graph.dot", "w");
    if (file == NULL) {
        printf("Erro ao criar o arquivo de plotagem.\n");
        return;
    }

    fprintf(file, "digraph PDA {\n");
    fprintf(file, "  rankdir=LR;\n");  // Direção do gráfico da esquerda para a direita
    fprintf(file, "  node [shape=circle];\n");

    // Estados
    for (int i = 0; i < automato.numEstados; ++i) {
        if (automato.estados[i].inicial) {
            fprintf(file, "  \"%s\" [shape=point];\n", "start");
            fprintf(file, "  \"%s\" -> \"%s\";\n", "start", automato.estados[i].nome);
        }
        fprintf(file, "  \"%s\" [shape=%s];\n", automato.estados[i].nome, automato.estados[i].final ? "doublecircle" : "circle");
    }

    // Transições
    for (int i = 0; i < automato.numTransicoes; ++i) {
        Transicao t = automato.transicoes[i];
        fprintf(file, "  \"%s\" -> \"%s\" [label=\"%c,%c/%c\"];\n",
                t.origem->nome,
                t.destino->nome,
                t.simbolo ? t.simbolo : 'ε',
                t.desempilha ? t.desempilha : '-',
                t.empilha ? t.empilha : '-');
    }

    fprintf(file, "}\n");
    fclose(file);
    printf("Arquivo de plotagem gerado: pda_graph.dot\n");
}
```

int leFita(char *fita):

Essa função é responsável por iniciar o processo de leitura e verificação de uma fita pelo autômato com pilha. Ela configura o ambiente necessário para essa verificação, como a inicialização da pilha, e chama a funçao recursiva leFitaRec() para realizar a verificação real.

```c
int leFita(char *fita) {
    inicializaPilha();
    return leFitaRec(automato.estadoInicial, fita, 0);
}
```

int aceita(char *fita):

Essa função é um invólucro para a função leFita(). Ela fornece uma interface clara e direta para verificar se uma fita é aceita pelo autômato.

```
int aceita(char *fita) {
    return leFita(fita);
}
```

int main():

Como essa função é a maior do código, vamos ir por partes. Primeiramente, ela inicializa o número de estados, estados finais e transições do autômato em zero. Em seguida declara as variáveis necessárias para armazenar as escolhas do usuário e dados temporários para estados, transições e a fita de entrada.

```
automato.numEstados = 0;
automato.numFinais = 0;
automato.numTransicoes = 0;

int opcao, inicial, final;
char nome[10], origem[10], destino[10], simbolo, desempilha, empilha;
char fita[MAX_FITA];
```

Continuando, ela cria um loop infinito que exibe o menu e processa a escolha do usuário até que a opção de sair seja selecionada. Usa um switch para determinar qual ação realizar com base na opção escolhida pelo usuário. Em seguida detalharemos o que ocorre em cada caso.

```
while (1) {
    printf("\nMenu:\n");
    printf("1. Criar Estado\n");
    printf("2. Criar Transição\n");
    printf("3. Mostrar Função Programa\n");
    printf("4. Verificar Palavra\n");
    printf("5. Gerar gráfico\n");
    printf("6. Sair\n");
    printf("Escolha uma opção: ");
    scanf("%d", &opcao);

    switch (opcao) {
```

Caso 1: Criar estado: Solicita ao usuário os dados do estado (nome, se é inicial, se é final) e chama a função CriaEstado() para adicionar o estado ao autômato.

```
case 1:
    printf("Nome do estado: ");
    scanf("%s", nome);
    printf("É inicial (1 para sim, 0 para não): ");
    scanf("%d", &inicial);
    printf("É final (1 para sim, 0 para não): ");
    scanf("%d", &final);
    CriaEstado(nome, inicial, final);
    break;
```

Caso 2: Cria transição: Solicita ao usuário os dados da transição (estado de origem, estado de destino, símbolo de transição, símbolo a desempilhar, símbolo a empilhar) e chama a função CriaTransicao() para adicionar a transição ao autômato.

```
case 2:
    printf("Estado de origem: ");
    scanf("%s", origem);
    printf("Estado de destino: ");
    scanf("%s", destino);
    printf("Símbolo de transição (use '-' para vazio): ");
    scanf(" %c", &simbolo);
    printf("Símbolo a desempilhar (use '-' para nenhum): ");
    scanf(" %c", &desempilha);
    if (desempilha == '-') desempilha = '\0';
    printf("Símbolo a empilhar (use '-' para nenhum): ");
    scanf(" %c", &empilha);
    if (empilha == '-') empilha = '\0';
    CriaTransicao(origem, destino, simbolo, desempilha, empilha);
    break;
```

Caso 3: Mostrar função programa: Chama a função mostraFuncPrograma para exibir todas as transições do autômato.

```
case 3:
    mostraFuncPrograma();
    break;
```

Caso 4: Verificar palavra: Solicita ao usuário uma palavra (fita) a ser verificada pelo autômato e usa a função aceita() para determinar se a palavra é aceita ou rejeitada.

```
case 4:
    printf("Digite a palavra a ser verificada: ");
    scanf("%s", fita);
    if (aceita(fita)) {
        printf("Palavra aceita.\n");
    } else {
        printf("Palavra rejeitada.\n");
    }
    break;
```

Caso 5: Gerar gráfico: Chama a função plot() para gerar um arquivo DOT (pda_graph.dot) que representa graficamente o autômato de pilha.

```
case 5:
    plot();
    break;
```

Caso 6: Sair: Encerra o programa.

```
case 6:
    exit(0);
    break;
```

E por fim, o caso default, que exibe uma mensagem caso a opção que o usuário escolheu seja inválida.

```
default:
    printf("Opção inválida!\n");
```

Aqui, o código da main por completo:

```c
int main() {
    automato.numEstados = 0;
    automato.numFinais = 0;
    automato.numTransicoes = 0;

    int opcao, inicial, final;
    char nome[10], origem[10], destino[10], simbolo, desempilha, empilha;
    char fita[MAX_FITA];

    while (1) {
        printf("\nMenu:\n");
        printf("1. Criar Estado\n");
        printf("2. Criar Transição\n");
        printf("3. Mostrar Função Programa\n");
        printf("4. Verificar Palavra\n");
        printf("5. Gerar gráfico\n");
        printf("6. Sair\n");
        printf("Escolha uma opção: ");
        scanf("%d", &opcao);

        switch (opcao) {
            case 1:
                printf("Nome do estado: ");
                scanf("%s", nome);
                printf("É inicial (1 para sim, 0 para não): ");
                scanf("%d", &inicial);
                printf("É final (1 para sim, 0 para não): ");
                scanf("%d", &final);
                CriaEstado(nome, inicial, final);
                break;
            case 2:
                printf("Estado de origem: ");
                scanf("%s", origem);
                printf("Estado de destino: ");
                scanf("%s", destino);
                printf("Símbolo de transição (use '-' para vazio): ");
                scanf(" %c", &simbolo);
                printf("Símbolo a desempilhar (use '-' para nenhum): ");
                scanf(" %c", &desempilha);
                if (desempilha == '-') desempilha = '\0';
                printf("Símbolo a empilhar (use '-' para nenhum): ");
                scanf(" %c", &empilha);
                if (empilha == '-') empilha = '\0';
                CriaTransicao(origem, destino, simbolo, desempilha, empilha);
                break;
            case 3:
                mostraFuncPrograma();
                break;
```

```
            case 4:
                printf("Digite a palavra a ser verificada: ");
                scanf("%s", fita);
                if (aceita(fita)) {
                    printf("Palavra aceita.\n");
                } else {
                    printf("Palavra rejeitada.\n");
                }
                break;
            case 5:
                plot();
                break;
            case 6:
                exit(0);
                break;
            default:
                printf("Opção inválida!\n");
        }
    }

    return 0;
}
```